Tecnológico de Estudios Superiores de Villa Guerrero
Ingeniería en Sistemas Computacionales

Joshua Villa González

Manual de Ejercicios de Evaluación
Punto de Equilibrio, Estadística Descriptiva y Regresión Lineal

Documentación de los scrips

Metodos Cuantitativos Aplicado a las Finanzas Para La Toma De Decisiones

Juan Reynoso Elias

3711

Índice general

Resumen vi

1 Introducción 1

1.1 Propósito . . . 1

1.2 Alcance . . . 1

1.3 Estructura del proyecto . . . 1

1.4 Entorno de ejecución . . . 2

1.5 Convenciones del documento . . . 3

2 Ejercicio 1: punto de equilibrio 3

2.1 Enunciado . . . 3

2.2 Fundamento teórico . . . 3

2.3 Aplicación con los datos del enunciado . . . 4

2.4 Descripción del script . . . 5

2.5 Forma de ejecución . . . 6

2.6 Representación gráfica . . . 6

2.7 Observaciones . . . 7

3 Ejercicio 2: estadística descriptiva 7

3.1 Enunciado . . . 7

3.2 Fundamento teórico . . . 8

3.3 Datos y tratamiento de valores faltantes . . . 8

3.4 Scripts de la variante CSV . . . 9

3.4.1 Script ejercicio-2-csv.py . . . 9

3.4.2 Script analizar.R 10
3.5 Componente de base de datos PostgreSQL 11
3.5.1 Archivo base_datos_postgres.sql 11
3.5.2 Script ejercicio-2-bd.py 12
3.5.3 Script analizar_db.R 13
3.6 Resultados 13
3.7 Interpretación 14
3.8 Observaciones 15
4 Ejercicio 3: regresión lineal simple 15
4.1 Enunciado 15
4.2 Fundamento teórico 16
4.3 Descripción del script 17
4.4 Forma de ejecución 18
4.5 Resultados 18
4.5.1 Modelo con pies cuadrados 19
4.5.2 Modelo con dormitorios 19
4.5.3 Modelo con antigüedad 19
4.6 Observaciones 20
5 Ejercicio 4: regresión lineal múltiple con pies cuadrados y dormitorios 21
5.1 Enunciado 21
5.2 Fundamento teórico 21
5.3 Descripción del script 22
5.4 Forma de ejecución 23
5.5 Resultados 23
5.6 Interpretación 23
5.7 Observaciones 25
6 Ejercicio 5: regresión lineal múltiple con tres predictores 26
6.1 Enunciado 26
6.2 Fundamento teórico 26
6.3 Descripción del script 26
6.4 Forma de ejecución 27
6.5 Resultados 27

6.6 Interpretación 28
6.7 Observaciones 29

7 Comparación, conclusiones y limitaciones **30**
7.1 Comparación de los modelos 30
7.2 Correspondencia entre el enunciado y la implementación 30
7.3 Conclusiones 31
7.4 Limitaciones y recomendaciones 31

Referencias **32**

A Código fuente completo **33**
A.1 Ejercicio 1: ejercicio-1.py 33
A.2 Ejercicio 2: ejercicio-2-csv.py 33
A.3 Ejercicio 2: analizar.R 34
A.4 Ejercicio 2: ejercicio-2-bd.py 34
A.5 Ejercicio 2: analizar_db.R 36
A.6 Ejercicio 2: esquema de base_datos_postgres.sql 39
A.7 Ejercicio 3: ejercicio-3.R 39
A.8 Ejercicio 4: ejercicio-4.R 40
A.9 Ejercicio 5: ejercicio-5.R 41

B Guía de edición y compilación **41**
B.1 Archivos del manual 41
B.2 Compilación 42
B.3 Sustitución de datos de portada 42
B.4 Logotipo institucional 43
B.5 Actualización de resultados 43
B.6 Dependencias de LaTeX 43
B.7 Posibles problemas 44

Índice de figuras

1 Punto de equilibrio del Ejercicio 1: intersección de las rectas de ingreso total y costo total. 7

2 Distribución del tipo de cambio por sexenio, elaborada a partir de los archivos CSV del Ejercicio 2. 15

3 Salida de R para los tres modelos de regresión lineal simple del Ejercicio 3. 20

4 Salida de R para el modelo de regresión múltiple y la predicción del Ejercicio 4. 25

5 Salida de R para el modelo de regresión múltiple y la predicción del Ejercicio 5. 29

Índice de tablas

1 Estructura de carpetas y archivos del proyecto. 2

2 Software utilizado para la ejecución y verificación de los scripts. 2

3 Ingreso, costo y utilidad para tres niveles de ventas. 5

4 Registros contenidos en los archivos del Ejercicio 2. 9

5 Estadísticas descriptivas del tipo de cambio por sexenio. 14

6 Datos de viviendas del Ejercicio 3. 16

7 Coeficientes de los modelos de regresión lineal simple del Ejercicio 3. 18

8 Bondad de ajuste de los modelos simples del Ejercicio 3. 19

9 Coeficientes del modelo de regresión múltiple del Ejercicio 4. 23

10 Resumen del modelo del Ejercicio 4. 23

11 Coeficientes del modelo de regresión múltiple del Ejercicio 5. 27

12 Resumen del modelo del Ejercicio 5. 27

13 Comparación de los modelos de regresión ajustados. 30

14 Correspondencia entre lo solicitado y lo implementado. 30

Resumen

El presente manual documenta la teoría, el funcionamiento y la forma de ejecución de los cinco ejercicios de evaluación incluidos en el proyecto. Los ejercicios abarcan el cálculo del punto de equilibrio, la estadística descriptiva de dos series de tipo de cambio y la estimación de precios de vivienda mediante regresión lineal simple y múltiple.

Para cada ejercicio se presenta el enunciado, el fundamento teórico, la descripción del script, las instrucciones de ejecución y los resultados obtenidos al ejecutar los programas en el entorno disponible. Además, se incluyen tablas, figuras, el código fuente completo en los apéndices y una guía para editar y recompilar este documento.

1 Introducción

1.1 Propósito

Este documento tiene como finalidad servir como manual de referencia para comprender, ejecutar y revisar los scripts desarrollados en los cinco ejercicios de evaluación. No se limita a mostrar los resultados: explica la teoría que sustenta cada cálculo, describe la lógica interna de cada programa y señala las condiciones necesarias para reproducir las salidas.

El manual fue redactado a partir de los archivos que integran el proyecto y de los enunciados transcritos en transcripci_n_de_ejercicios.txt. Los valores numéricos presentados en las tablas y en el texto corresponden a la ejecución de los scripts incluidos, salvo que se indique expresamente lo contrario.

1.2 Alcance

Se documentan los siguientes componentes:

- El script de punto de equilibrio del Ejercicio 1.
- Los scripts de estadística descriptiva del Ejercicio 2, tanto en la variante de archivos CSV como en la variante de base de datos PostgreSQL.
- Los scripts de regresión lineal simple del Ejercicio 3.
- Los scripts de regresión lineal múltiple de los Ejercicios 4 y 5.
- El archivo SQL que crea y carga la base de datos utilizada por los scripts de la variante PostgreSQL.

No se modificó la lógica de los scripts. Las observaciones sobre posibles diferencias con el enunciado se presentan de forma explícita en la sección correspondiente y en la comparación final.

1.3 Estructura del proyecto

La Tabla 1 resume la organización de las carpetas y los archivos empleados.

Tabla 1: Estructura de carpetas y archivos del proyecto.

Carpeta	Archivos principales	Función
ejercicio-1	ejercicio-1.py; ejercicio-1.png	Cálculo del punto de equilibrio y captura de una variante del script.
ejercicio-2	2.xls; sexenio-1.xls; sexenio-2.xls; sexenio-1.csv; sexenio-2.csv; ejercicio-2-csv.py; analizar.R; base_datos_postgres.sql; ejercicio-2-bd.py; analizar_db.R	Estadística descriptiva de los tipos de cambio, por CSV y por PostgreSQL.
ejercicio-3	ejercicio-3.R; ejercicio-3.png	Tres modelos de regresión simple.
ejercicio-4	ejercicio-4.R; ejercicio-4.png	Regresión múltiple con pies y dormitorios.
ejercicio-5	ejercicio-5.R; ejercicio-5.png	Regresión múltiple con pies, dormitorios y antigüedad.
manual	manual.tex; compilar.sh; apoyo e generar_boxplot.py; imagenes/ manual.	Fuente LaTeX, scripts de imágenes del

1.4 Entorno de ejecución

Los scripts fueron ejecutados con las versiones de software indicadas en la Tabla 2. Las versiones pueden variar en otro equipo, pero los resultados numéricos de los scripts incluidos no dependen de cambios menores del entorno.

Tabla 2: Software utilizado para la ejecución y verificación de los scripts.

Herramienta	Versión verificada	Uso
Python	3.14.8	Ejercicios 1 y 2 (variantes CSV y base de datos).
R	4.6.1	Ejercicios 2, 3, 4 y 5.
PostgreSQL	18.6	Base de datos del Ejercicio 2.
psql	18.6	Cliente de línea de comandos para PostgreSQL.
pdfTeX (TeX Live)	3.141592653-2.6-1.40.29	Compilación de este manual.

Los scripts de Python utilizan únicamente la biblioteca estándar (csv, statistics, os, shutil y subprocess). Los scripts de R emplean funciones base

como read.csv, lm, summary, predict y system2. La variante de base de datos requiere que el servidor PostgreSQL esté en ejecución y que el cliente psql esté disponible en el PATH.

1.5 Convenciones del documento

- Los textos entre corchetes, como [COMPLETAR], son campos que deben sustituirse antes de la entrega final.
- Los resultados numéricos se transcriben con el punto decimal que emplean Python y R, con el fin de conservar la correspondencia exacta con las salidas de los programas.
- Los comandos se presentan en bloques de consola y se ejecutan desde la carpeta indicada en cada caso.
- Las rutas de archivo son relativas a la carpeta examen, que contiene las carpetas de los cinco ejercicios y la carpeta manual.
- Las figuras y tablas se numeran por sección y se referencian en el texto mediante sus etiquetas.

2 Ejercicio 1: punto de equilibrio

2.1 Enunciado

Una empresa vende un producto a $40 USD por unidad. Los costos fijos mensuales son de $3,500 USD. Los costos variables por unidad constan de
$24 USD de producción y $2 USD de comisión por venta.

1. Formule el modelo para el punto de equilibrio en Excel.
2. Determine el punto de equilibrio en unidades e ingrese la cantidad obtenida.

El script incluido en la carpeta ejercicio-1 implementa en Python el cálculo del punto de equilibrio. El enunciado también solicita formular el modelo en Excel; en la carpeta no se incluye un archivo de Excel, por lo que este manual documenta la implementación disponible.

2.2 Fundamento teórico

En un modelo lineal de costos, se definen las siguientes variables:

- s: precio de venta por unidad.
- F: costos fijos totales del periodo.

- v: costo variable por unidad.
- x: cantidad de unidades producidas y vendidas.

El ingreso total y el costo total se expresan como:

$$I(x) = sx \quad \text{y} \quad C(x) = F + v\,x. \tag{1}$$

El punto de equilibrio es la cantidad x para la cual el ingreso total iguala al costo total, es decir, $I(x) = C(x)$. Al resolver la ecuación se obtiene:

$$sx = F + v\,x \implies (s - v)\,x = F \implies x_{\text{eq}} = \frac{F}{s - v}. \tag{2}$$

La diferencia $s - v$ se denomina margen de contribución unitario. Si $s > v$, el margen es positivo y existe un punto de equilibrio finito. Si $s \leq v$, el margen es cero o negativo y la empresa no puede cubrir los costos fijos aumentando las ventas dentro de ese modelo.

La utilidad se define como:

$$U(x) = I(x) - C(x) = (s - v)\,x - F. \tag{3}$$

En el punto de equilibrio, $U(x_{\text{eq}}) = 0$. Para $x > x_{\text{eq}}$ la utilidad es positiva; para $x < x_{\text{eq}}$ la utilidad es negativa.

2.3 Aplicación con los datos del enunciado

Los datos del enunciado son $s = 40$ USD por unidad, $F = 3{,}500$ USD y un costo variable unitario compuesto por \$24 de producción y \$2 de comisión, es decir:

$$v = 24 + 2 = 26 \text{ USD por unidad.} \tag{4}$$

) Al sustituir en la ecuación (2):

$$x_{\text{eq}} = \frac{3{,}500}{40 - 26} = \frac{3{,}500}{14} = 250 \text{ unidades.} \tag{5}$$

La Tabla 3 ilustra el comportamiento de la utilidad alrededor del

punto de equilibrio. Los valores se obtienen directamente de las ecuaciones (1) y (3).

Tabla 3: Ingreso, costo y utilidad para tres niveles de ventas.

Unidades (x)	Ingreso $I(x)$	Costo $C(x)$	Utilidad $U(x)$	Situación
200	8,000	8,700	−700	Pérdida
250	10,000	10,000	0	Equilibrio
300	12,000	11,300	700	Ganancia

Resultado

Con los datos del enunciado, el punto de equilibrio es de **250 unidades**. En ese nivel, el ingreso total y el costo total son ambos de $10,000 USD.

2.4 Descripción del script

El archivo ejercicio-1.py contiene el siguiente código:

```
s = int(input("Ingresa el precio por unidad: "))
f = int(input("Ingresa el costo fijo: "))
v = int(input("Ingresa el costo variable por unidad: "))

pequilibrio = f/(s-v)
print("El punto de equilibrio es: ", pequilibrio)
```

Listado 1: Script completo del Ejercicio 1.

El funcionamiento es el siguiente:

1. `s = int(input(...))`: solicita al usuario el precio de venta por unidad y lo convierte a un número entero.
2. `f = int(input(...))`: solicita el costo fijo total y lo convierte a entero.
3. `v = int(input(...))`: solicita el costo variable por unidad y lo convierte a entero.
4. `pequilibrio = f/(s-v)`: aplica la ecuación (2). En Python, el operador `/` produce un resultado de tipo float, incluso si el cociente es exacto.
5. `print(...)`: muestra en pantalla el valor calculado.

El script no valida que $s > v$, no comprueba que las entradas sean numéricas y no calcula la utilidad. Su alcance es exclusivamente el punto de equilibrio.

2.5 Forma de ejecución

Desde la carpeta ejercicio-1, ejecutar:

```
cd ejercicio-1
python3 ejercicio-1.py
```

Listado 2: Ejecución del script del Ejercicio 1.

El programa solicita tres valores. Para reproducir el resultado del enunciado se introducen:

```
Ingresa el precio por unidad: 40
Ingresa el costo fijo: 3500
Ingresa el costo variable por unidad: 26
El punto de equilibrio es:  250.0
```

Listado 3: Entradas y salida esperada del Ejercicio 1.

> **Nota**
>
> El costo variable que debe introducirse es **26**, no 24, porque el enunciado indica que el costo variable unitario incluye $24 de producción y $2 de comisión por venta. El script recibe un único costo variable por unidad.

2.6 Representación gráfica

La Figura 1 muestra las rectas de ingreso total y costo total. El punto de intersección corresponde al punto de equilibrio calculado.

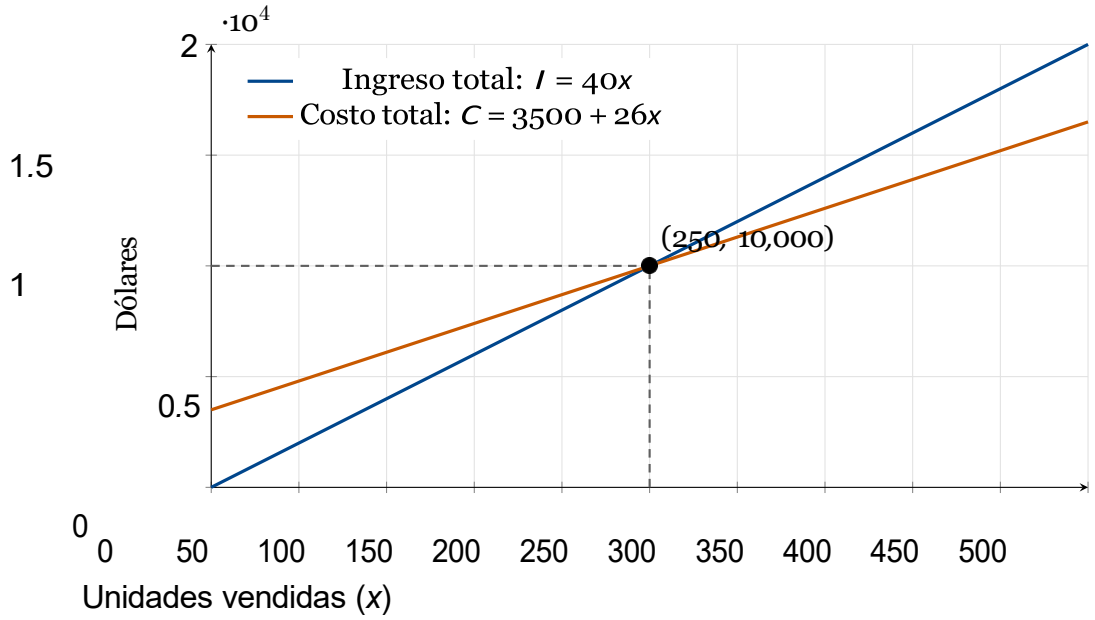


Figura 1: Punto de equilibrio del Ejercicio 1: intersección de las rectas de ingreso total y costo total.

2.7 Observaciones

- La captura ejercicio-1.png incluida en la carpeta muestra una variante del script que también solicita el número de unidades vendidas y calcula la ganancia. Esa variante no corresponde al archivo ejercicio-1.py documentado en este manual, que solo calcula el punto de equilibrio.
- El uso de `int()` obliga a introducir números enteros. Si se requieren precios o costos con decimales, sería necesario emplear `float()`; sin embargo, el script entregado no lo hace.
- Si el usuario introduce un costo variable mayor o igual que el precio, el denominador $s - v$ puede ser cero o negativo. El script no controla esa situación.

3 Ejercicio 2: estadística descriptiva

3.1 Enunciado

En los archivos sexenio-1.xls y sexenio-2.xls se encuentran los tipos de cambio registrados en dos periodos de tiempo. Utilizando el lenguaje R y/o Python, determine las siguientes métricas para cada conjunto de datos: media, varianza, desviación estándar, máximo,

mínimo y rango. Presente sus resultados organizados en una tabla.

3.2 Fundamento teórico

Para un conjunto de N observaciones $x_1, x_2, \ldots, x_N$, la media aritmética se define como:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i. \tag{6}$$

La varianza mide la dispersión promedio de los datos respecto de la media. Los scripts incluidos emplean la varianza poblacional:

$$\sigma_N^2 = \frac{1}{N} \sum_{i=1}^{N} (x_i - \bar{x})^2. \tag{7}$$

La desviación estándar es la raíz cuadrada de la varianza:

$$\sigma = \sqrt{\sigma^2}. \tag{8}$$

El máximo y el mínimo son el mayor y el menor valor observado, respectivamente.

El rango se define como:

$$R = x_{\max} - x_{\min}. \tag{9}$$

Nota

Los scripts de Python usan `statistics.pvariance` y `statistics.pstdev`, que calculan la varianza y la desviación estándar **poblacionales** (divisor N). El script de R calcula $\frac{1}{N}\sum (x_i - \bar{x})^2$, que es la misma definición. Si se desea la varianza muestral, debe usarse el divisor $N - 1$; los scripts entregados no lo hacen.

3.3 Datos y tratamiento de valores faltantes

La carpeta `ejercicio-2` contiene los archivos originales `sexenio-1.xls` y `sexenio-2.xls`, así como los archivos `sexenio-1.csv` y `sexenio-2.csv`. Los scripts documentados leen los archivos CSV o consultan la base de datos PostgreSQL; no abren directamente los archivos `.xls`.

Los CSV tienen dos columnas: Fecha y Para solventar obligaciones. La segunda columna contiene el tipo de cambio. Algunos registros del primer sexenio aparecen como N/E; esos valores se excluyen del cálculo. La Tabla 4 resume la cantidad de registros.

Tabla 4: Registros contenidos en los archivos del Ejercicio 2.

Archivo	Registros totales	Registros válidos	Valores N/E
sexenio-1.csv	2,557	2,516	41
sexenio-2.csv	2,192	2,192	0

En la base de datos, el primer sexenio conserva 2,557 filas, de las cuales 41 tienen tipo_cambio nulo; el segundo sexenio contiene 2,192 filas con valor no nulo. Por tanto, los registros efectivamente utilizados coinciden con los de la tabla anterior.

3.4 Scripts de la variante CSV

3.4.1 Script ejercicio-2-csv.py

Este script recorre los dos archivos CSV, omite los valores N/E y calcula las métricas con la biblioteca estándar de Python. Su código completo se reproduce en el Apéndice A. El núcleo del procesamiento es:

```
RUTAS = ("./sexenio-1.csv", "./sexenio-2.csv")

for ruta in RUTAS:
    with open(ruta, newline="", encoding="utf-8") as archivo:
        lector = csv.reader(archivo)
        next(lector)
        datos = [float(fila[1]) for fila in lector if fila[1] != "N/E"]

    print(f"Media: {statistics.mean(datos):.4f}")
    print(f"Varianza: {statistics.pvariance(datos):.4f}")
    print(f"Desviación estándar: {statistics.pstdev(datos):.4f}")
    print(f"Máximo: {max(datos):.4f}")
    print(f"Mínimo: {min(datos):.4f}")
    print(f"Rango: {max(datos) - min(datos):.4f}")
```

Listado 4: Fragmento principal de ejercicio-2-csv.py.

El funcionamiento es el siguiente:

1. RUTAS define los dos archivos que se procesarán.

2. open(..., newline="", encoding="utf-8") abre cada archivo respetando los saltos de línea y la codificación UTF-8.

3. csv.reader permite leer el archivo fila por fila.
4. next(lector) descarta la fila de encabezado.
5. La lista por comprensión convierte a float la segunda columna de cada fila y omite los valores N/E.
6. Las funciones statistics.mean, statistics.pvariance y statistics.pstdev calculan la media, la varianza poblacional y la desviación estándar poblacional.
7. max, min y la resta correspondiente calculan el máximo, el mínimo y el rango.
8. El formato :.4f presenta los resultados con cuatro cifras decimales.

Para ejecutarlo, desde la carpeta ejercicio-2:

```
python3 ejercicio-2-csv.py
```

Listado 5: Ejecución de la variante CSV en Python.

3.4.2 Script analizar.R

Este script realiza el mismo cálculo en R. Lee cada CSV, convierte la columna de tipo de cambio a numérico, elimina los valores faltantes y aplica las fórmulas de forma explícita. Su código completo se reproduce en el Apéndice A. El fragmento principal es:

```
datos <- suppressWarnings(
  as.numeric(tabla[["Para solventar obligaciones"]])
)
datos <- datos[!is.na(datos)]

media <- mean(datos)
varianza <- mean((datos - media)^2)
desviacion <- sqrt(varianza)
maximo <- max(datos)
minimo <- min(datos)
rango <- maximo - minimo
```

Listado 6: Fragmento principal de analizar.R.

El funcionamiento es el siguiente:

1. read.csv lee el archivo con la codificación UTF-8.
2. check.names = FALSE conserva el nombre original de la columna, incluidos los espacios.
3. as.numeric convierte la columna a números; los valores N/E se convierten en
   NA. suppressWarnings evita que R muestre la advertencia de conversión.
4. datos[!is.na(datos)] elimina los valores faltantes.
5. La varianza se calcula como el promedio de las diferencias al cuadrado respecto de la media, es decir, la varianza poblacional de la ecuación (7).
6. cat y sprintf presentan los resultados con cuatro decimales.

Para ejecutarlo:

```
Rscript analizar.R
```

Listado 7: Ejecución de la variante CSV en R.

3.5 Componente de base de datos PostgreSQL

3.5.1 Archivo base_datos_postgres.sql

Este archivo crea la base de datos tipo_cambio, el usuario de aplicación y las tablas necesarias. Su estructura principal es la siguiente:

```sql
CREATE TABLE sexenios (
    id INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE tipos_cambio (
    id SERIAL PRIMARY KEY,
    sexenio_id INTEGER NOT NULL REFERENCES sexenios(id),
    fecha DATE NOT NULL,
    tipo_cambio NUMERIC(10,4),
    UNIQUE (sexenio_id, fecha)
);
```

Listado 8: Esquema principal de la base de datos del Ejercicio 2.

La tabla sexenios almacena los dos periodos. La tabla tipos_cambio almacena una fila por fecha, con una llave foránea hacia sexenios y una restricción de unicidad sobre la pareja (sexenio_id, fecha). La columna tipo_cambio admite valores nulos, lo que permite representar los registros N/E como NULL.

El archivo también contiene las sentencias INSERT con los datos de ambos sexenios y los permisos de lectura para el usuario de aplicación. El encabezado del archivo indica el comando de carga:

```
sudo -u postgres psql < base_datos_postgres.sql
```

Listado 9: Carga de la base de datos desde el archivo SQL.

Advertencia

El archivo SQL ejecuta DROP DATABASE IF EXISTS y DROP ROLE IF EXISTS antes de crear la base y el usuario. Esto significa que elimina una base o un rol previos con el mismo nombre. Debe ejecutarse en un entorno controlado y no sobre una base de datos que contenga información que se desee conservar. Las credenciales de práctica están definidas en el propio archivo y en los scripts; no se reproducen aquí por razones de seguridad.

3.5.2 Script ejercicio-2-bd.py

Este script consulta la base de datos mediante el cliente psql, sin utilizar un conector de Python. El funcionamiento es el siguiente:

1. Define un diccionario DB con el host, puerto, usuario, contraseña y nombre de la base. Los valores se toman de las variables de entorno DB_HOST, DB_PORT, DB_USER, DB_PASSWORD y DB_NAME; si no existen, se usan valores predeterminados.

2. Localiza el ejecutable de psql con shutil.which. Si no lo encuentra, termina con un mensaje de error.

3. La función consultar construye un comando de psql con salida sin formato, sin alineación y separada por tabuladores. La contraseña se pasa mediante la variable de entorno PGPASSWORD.

4. Ejecuta el comando con subprocess.run y captura la salida. Si el

código de retorno no es cero, muestra el error y termina.

5. Consulta la tabla sexenios, muestra las opciones y solicita al usuario el número del sexenio.
6. Consulta los tipos de cambio no nulos del sexenio elegido, ordenados por fecha, y calcula las métricas con las mismas funciones de la variante CSV.

Para ejecutarlo, después de cargar la base de datos:

```
python3 ejercicio-2-bd.py
```

Listado 10: Ejecución de la variante PostgreSQL en Python.

Las variables de entorno pueden ajustarse antes de la ejecución, por ejemplo:

```
export DB_HOST=localhost
export DB_PORT=5432
export DB_USER=tipo_cambio_app
export DB_PASSWORD='[COMPLETAR]'
export DB_NAME=tipo_cambio
python3 ejercicio-2-bd.py
```

Listado 11: Configuración opcional de la conexión a PostgreSQL.

3.5.3 Script analizar_db.R

Este script es el equivalente en R de la variante PostgreSQL. Localiza psql con Sys.which, construye los argumentos de conexión, ejecuta la consulta con system2 y procesa la salida separada por dos puntos. También guarda y restaura la variable PGPASSWORD mediante on.exit, para no dejar la contraseña modificada al finalizar. Su código completo se reproduce en el Apéndice A.

Para ejecutarlo:

```
Rscript analizar_db.R
```

Listado 12: Ejecución de la variante PostgreSQL en R.

3.6 Resultados

La Tabla 5 presenta los resultados obtenidos al ejecutar ejercicio-2-csv.py y
analizar.R. Ambos scripts producen los mismos valores.

Tabla 5: Estadísticas descriptivas del tipo de cambio por sexenio.

Métrica	Sexenio 2012–2018	Sexenio 2019–2024
Media	16.0351	19.5385
Varianza	7.9107	2.5603
Desviación estándar	2.8126	1.6001
Máximo	21.9076	25.1185
Mínimo	11.9807	16.3357
Rango	9.9269	8.7828

Nota

Los valores de la Tabla 5 se calcularon excluyendo los registros `N/E` del primer sexenio y utilizando la varianza poblacional. La variante de base de datos aplica las mismas fórmulas sobre los mismos registros; si la base se cargó con el archivo SQL incluido, su salida debe coincidir con esta tabla.

3.7 Interpretación

- La media del segundo sexenio (19.5385) es mayor que la del primero (16.0351). La diferencia aritmética es 19.5385 − 16.0351 = 3.5034 pesos por dólar.
- La varianza y la desviación estándar del segundo sexenio son menores (2.5603 y 1.6001) que las del primero (7.9107 y 2.8126). Esto indica una menor dispersión relativa de los valores alrededor de su media en el segundo periodo.
- El rango del primer sexenio (9.9269) es ligeramente mayor que el del segundo (8.7828), aunque el máximo del segundo sexenio (25.1185) es más alto que el del primero (21.9076). Esto es compatible con una distribución del segundo periodo más concentrada, pero con valores extremos altos.
- El mínimo del segundo sexenio (16.3357) es mayor que el del primero (11.9807), lo que refleja un nivel general más alto del tipo de cambio en el segundo periodo.

La Figura 2 presenta un diagrama de caja comparativo elaborado a partir de los archivos CSV. La caja central muestra el rango intercuartílico, la línea central la mediana, el triángulo la media y los puntos los valores atípicos.

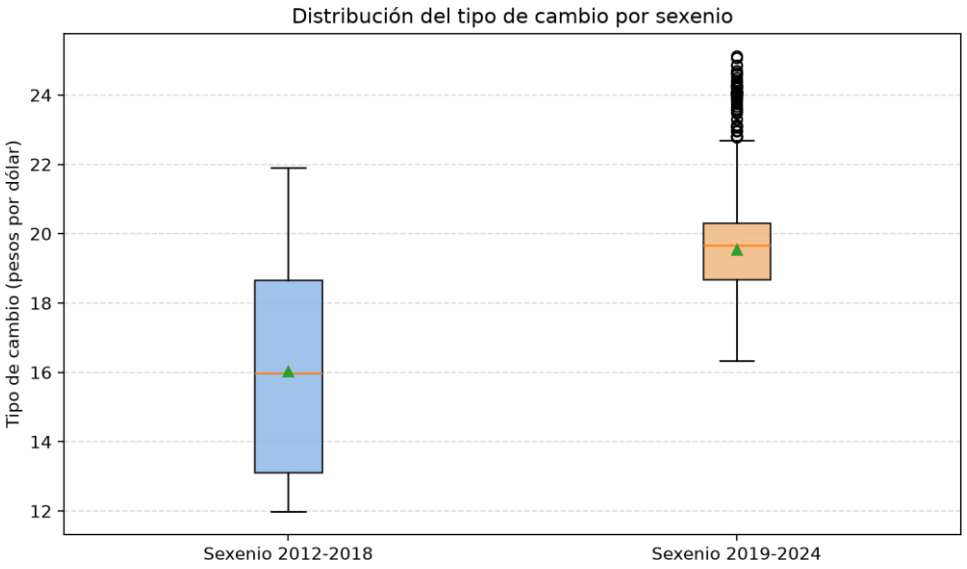


Figura 2: Distribución del tipo de cambio por sexenio, elaborada a partir de los archivos CSV del Ejercicio 2.

3.8 Observaciones

- Los scripts documentados trabajan con los archivos CSV y con PostgreSQL. No leen directamente los archivos .xls; si el enunciado exige usar los .xls, sería necesario añadir una lectura con una biblioteca específica o convertir los archivos.
- Los valores N/E se excluyen de tres formas equivalentes: filtrando la cadena en Python, eliminando NA en R y usando IS NOT NULL en SQL.
- La variante de base de datos requiere que el servicio PostgreSQL esté activo, que la base se haya cargado y que el usuario de aplicación tenga permisos de lectura sobre las tablas.
- La contraseña de práctica aparece en el archivo SQL y como valor predeterminado en los scripts. Para un entorno real, conviene usar variables de entorno y no almacenar credenciales en el código.

4 Ejercicio 3: regresión lineal simple

4.1 Enunciado

Un agente inmobiliario desea estimar el precio de venta de casas basadas

en algunas de sus características. Para ello, ha recopilado información histórica

sobre las ventas realizadas en los últimos 6 meses en su zona. Los datos recolectados se muestran en la tabla.

El ejercicio proporciona 17 observaciones de precio de venta, pies cuadrados, número de dormitorios y antigüedad. Los datos se reproducen en la Tabla 6.

Tabla 6: Datos de viviendas del Ejercicio 3.

Precio de venta ($)	Pies cuadrados	Dormitorios	Antigüedad (años)
64,000	1,670	2	30
59,000	1,339	2	25
61,500	1,712	3	30
79,000	1,840	3	40
87,500	2,300	3	18
92,500	2,234	3	30
95,000	2,311	3	19
113,000	2,377	3	7
115,000	2,736	4	10
138,000	2,500	3	1
142,500	2,500	4	3
144,000	2,479	3	3
145,000	2,400	3	1
147,500	3,124	4	0
144,000	2,500	3	2
155,500	4,062	4	10
165,000	2,854	3	3

4.2 Fundamento teórico

La regresión lineal simple modela la relación entre una variable dependiente y y una variable independiente x mediante:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad i = 1, 2, \ldots, n, \tag{10}$$

donde β_0 es el intercepto, β_1 es la pendiente y ε_i es el error aleatorio. El método de mínimos cuadrados ordinarios (MCO) elige los estimadores $\hat{\beta}_0$ y $\hat{\beta}_1$ que minimizan la suma de cuadrados de los residuos:

$$\text{SCE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2. \tag{11}$$

El modelo ajustado se escribe como $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$. La pendiente estimada indica el cambio promedio en *y* asociado a un aumento de una unidad en *x*. El intercepto indica el valor promedio estimado de *y* cuando *x* = 0, siempre que ese valor tenga

sentido dentro del contexto y del rango observado.

La bondad de ajuste se resume con el coeficiente de determinación:

$$R^2 = 1 - \frac{\mathrm{SCE}}{\mathrm{SCT}}, \quad \mathrm{SCT} = \sum_{i=1} (y_i - \bar{y})^2. \tag{12}$$

El R^2 indica la proporción de la variabilidad de y explicada por el modelo. El R^2 ajustado penaliza la inclusión de predictores y se calcula como:

$$R^2_{\mathrm{aj}} = 1 - \frac{(1 - R^2)(n - 1)}{n - k - 1}, \tag{13}$$

donde k es el número de predictores. Para una regresión simple, $k = 1$.

La significancia individual de la pendiente se contrasta con la hipótesis nula $H_0 : \beta_1 = 0$ frente a la alternativa $H_1 : \beta_1 \neq 0$. El estadístico de prueba es $t = \hat{\beta}_1 / \mathrm{EE}(\hat{\beta}_1)$, y el valor p indica la probabilidad de observar un resultado al menos tan extremo si la hipótesis nula fuera cierta. Un valor p pequeño se interpreta como evidencia contra H_0.

El error estándar residual estima la desviación típica de los errores del modelo. Los supuestos habituales para la inferencia son linealidad, independencia de los errores, homocedasticidad y normalidad de los errores. El script del Ejercicio 3 ajusta los modelos y muestra los resúmenes, pero no incluye gráficos de diagnóstico ni pruebas formales de esos supuestos.

4.3 Descripción del script

El archivo ejercicio-3.R define cuatro vectores con los datos, construye un data.frame y ajusta tres modelos de regresión lineal simple:

```r
datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_pies <- lm(precio ~ pies, data = datos)
summary(modelo_pies)

modelo_dorm <- lm(precio ~ dormitorios, data = datos)
summary(modelo_dorm)

modelo_antiguedad <- lm(precio ~ antiguedad, data = datos)
summary(modelo_antiguedad)
```

Listado 13: Modelos del Ejercicio 3.

El funcionamiento es el siguiente:

1. Los vectores precio, pies, dormitorios y antiguedad contienen las 17 observaciones en el mismo orden.
2. data.frame reúne los vectores en una tabla con nombres de columna.
3. lm(precio ~ pies, data = datos) ajusta el modelo de precio en función de los pies cuadrados.
4. lm(precio ~ dormitorios, data = datos) ajusta el modelo de precio en función del número de dormitorios.
5. lm(precio ~ antiguedad, data = datos) ajusta el modelo de precio en función de la antigüedad.
6. summary muestra los coeficientes estimados, los errores estándar, los estadísticos

t, los valores p, el R^2, el R^2 ajustado, el error estándar residual y la prueba F.

4.4 Forma de ejecución

Desde la carpeta ejercicio-3:

```
Rscript ejercicio-3.R
```

Listado 14: Ejecución del script del Ejercicio 3.

La salida completa se muestra en la Figura 3. También puede ejecutarse el archivo dentro de una sesión interactiva de R con source("ejercicio-3.R").

4.5 Resultados

La Tabla 7 resume los coeficientes de los tres modelos simples. La Tabla 8 presenta las medidas de bondad de ajuste.

Tabla 7: Coeficientes de los modelos de regresión lineal simple del Ejercicio 3.

Modelo	Término	Estimación	Error est.	Valor t	Valor p
Precio ~ pies	Intercepto	2,367.261	22,118.785	0.107	0.916188
	Pies	46.601	8.912	5.229	0.000102
Precio ~ dormitorios	Intercepto	1,924	39,028	0.049	0.9613
	Dormitorios	36,138	12,306	2.937	0.0102
Precio ~ antigüedad	Intercepto	147,670.9	6,247.0	23.639	2.77×10^{-13}
	Antigüedad	−2,424.2	335.6	−7.224	2.95×10^{-6}

Intercept

o

Tabla 8: Bondad de ajuste de los modelos simples del Ejercicio 3.

Modelo ajustado	R^2	R^2	Error est. residual	p de la prueba F
Precio ~ pies	0.6457	0.6221	22,070	0.0001019
Precio ~ dormitorios	0.3651	0.3227	29,550	0.01021
Precio ~ antigüedad		0.7768	17,520	2.953×10^{-6}
	0.7619			

4.5.1 Modelo con pies cuadrados

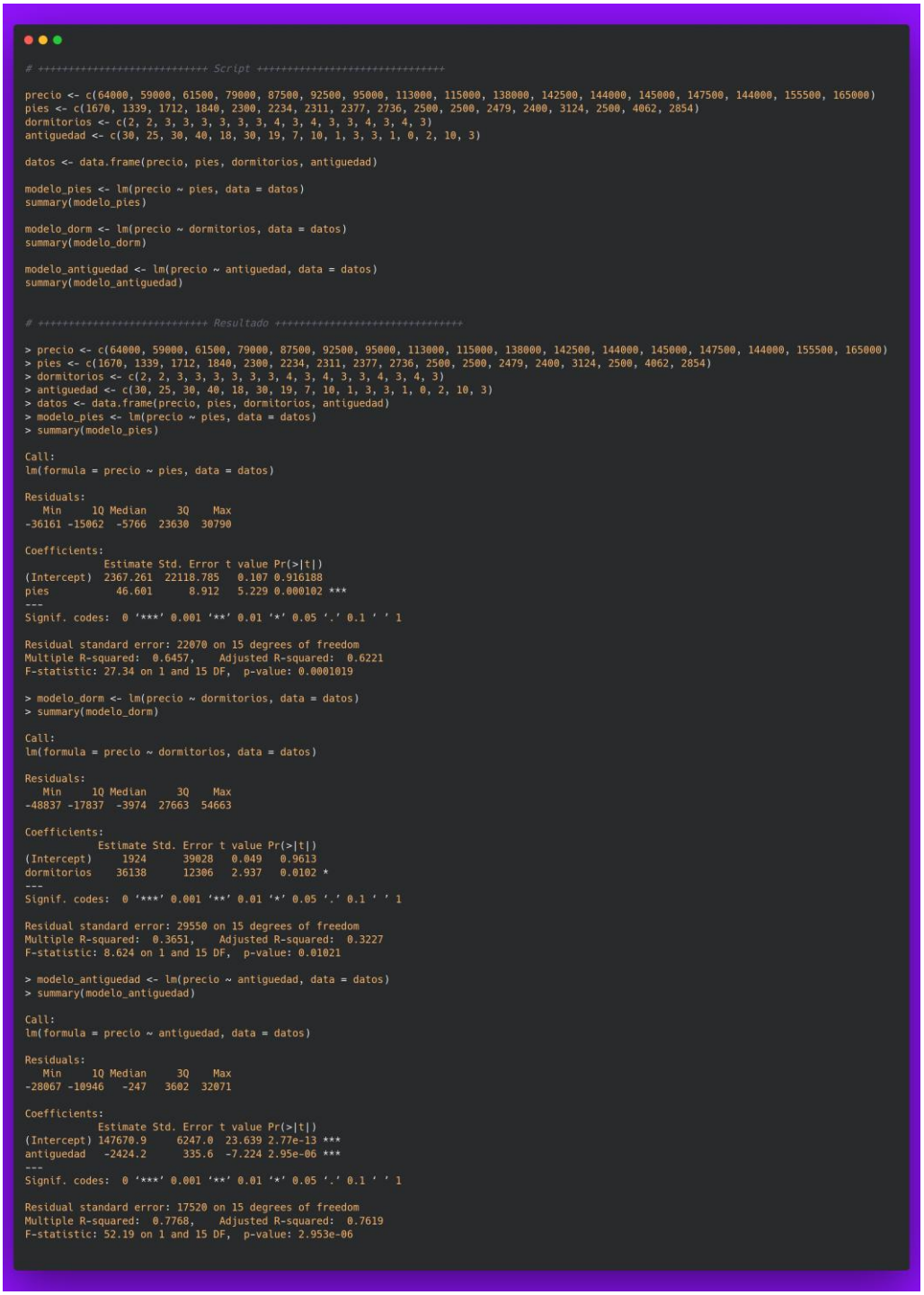
La pendiente estimada es 46.601. Esto significa que, en este modelo simple, un pie cuadrado adicional se asocia con un aumento promedio de $46.601 USD en el precio de venta. El valor p = 0.000102 es menor que 0.001, por lo que existe evidencia estadística de una relación lineal positiva. El R^2 = 0.6457 indica que el modelo explica aproximadamente el 64.57% de la variabilidad del precio. El intercepto estimado (2,367.261) no es estadísticamente significativo (p = 0.916188) y corresponde a una extrapolación fuera del rango de datos, ya que una vivienda de cero pies cuadrados no forma parte del conjunto observado.

4.5.2 Modelo con dormitorios

La pendiente estimada es 36,138. En este modelo simple, un dormitorio adicional se asocia con un aumento promedio de $36,138 USD en el precio. El valor p = 0.0102 es menor que 0.05, por lo que la relación es significativa al 5%. Sin embargo, el R^2 = 0.3651 es el más bajo de los tres modelos y la variable solo toma valores 2, 3 y 4, por lo que la estimación se apoya en un rango muy limitado. El intercepto (1,924) no es significativo (p = 0.9613).

4.5.3 Modelo con antigüedad

La pendiente estimada es −2,424.2. En este modelo simple, un año adicional de antigüedad se asocia con una disminución promedio de $2,424.2 USD en el precio. El valor $p = 2.95 \times 10^{-6}$ es muy pequeño, por lo que la relación negativa es estadísticamente significativa. El R^2 = 0.7768 indica que la antigüedad explica aproximadamente el 77.68% de la variabilidad del precio, el mayor porcentaje entre los tres modelos simples. El intercepto (147,670.9) es significativo y representa el precio promedio estimado para una vivienda nueva (0 años), valor que sí aparece en los datos.



```
# ++++++++++++++++++++++++++ Script ++++++++++++++++++++++++++++

precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_pies <- lm(precio ~ pies, data = datos)
summary(modelo_pies)

modelo_dorm <- lm(precio ~ dormitorios, data = datos)
summary(modelo_dorm)

modelo_antiguedad <- lm(precio ~ antiguedad, data = datos)
summary(modelo_antiguedad)


# ++++++++++++++++++++++++++ Resultado ++++++++++++++++++++++++++++

> precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
> pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
> dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
> antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)
> datos <- data.frame(precio, pies, dormitorios, antiguedad)
> modelo_pies <- lm(precio ~ pies, data = datos)
> summary(modelo_pies)

Call:
lm(formula = precio ~ pies, data = datos)

Residuals:
   Min     1Q Median     3Q    Max
-36161 -15062  -5766  23630  30790

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  2367.261  22118.785   0.107 0.916188
pies           46.601      8.912   5.229 0.000102 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 22070 on 15 degrees of freedom
Multiple R-squared:  0.6457,    Adjusted R-squared:  0.6221
F-statistic: 27.34 on 1 and 15 DF,  p-value: 0.0001019

> modelo_dorm <- lm(precio ~ dormitorios, data = datos)
> summary(modelo_dorm)

Call:
lm(formula = precio ~ dormitorios, data = datos)

Residuals:
   Min     1Q Median     3Q    Max
-48837 -17837  -3974  27663  54663

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)     1924      39028   0.049   0.9613
dormitorios    36138      12306   2.937   0.0102 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 29550 on 15 degrees of freedom
Multiple R-squared:  0.3651,    Adjusted R-squared:  0.3227
F-statistic: 8.624 on 1 and 15 DF,  p-value: 0.01021

> modelo_antiguedad <- lm(precio ~ antiguedad, data = datos)
> summary(modelo_antiguedad)

Call:
lm(formula = precio ~ antiguedad, data = datos)

Residuals:
   Min     1Q Median     3Q    Max
-28067 -10946   -247   3602  32071

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 147670.9     6247.0  23.639 2.77e-13 ***
antiguedad   -2424.2      335.6  -7.224 2.95e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 17520 on 15 degrees of freedom
Multiple R-squared:  0.7768,    Adjusted R-squared:  0.7619
F-statistic: 52.19 on 1 and 15 DF,  p-value: 2.953e-06
```

Figura 3: Salida de R para los tres modelos de regresión lineal simple del Ejercicio 3.

4.6 Observaciones

- El script ajusta tres modelos separados, no un modelo múltiple. Cada pendiente debe interpretarse sin controlar las demás características.

- La variable dormitorios es discreta y tiene poca variación en la muestra; esto limita la precisión de su efecto estimado en un modelo simple.
- El script no presenta intervalos de confianza ni gráficos de residuos. Para una revisión más completa de los supuestos, sería necesario añadirlos.
- El precio está expresado en dólares y las variables explicativas en pies cuadrados, dormitorios y años, respectivamente. Las unidades de los coeficientes corresponden a dólares por unidad de cada predictor.

5 Ejercicio 4: regresión lineal múltiple con pies cuadrados y dormitorios

5.1 Enunciado

Con base en la información recopilada en el Ejercicio 3:

1. *Ajuste un modelo de regresión lineal múltiple en Python para estimar el precio de venta en función de los pies cuadrados y el número de dormitorios.*
2. *Utilice el modelo ajustado para estimar el precio de venta de una casa que tiene 2,000 pies cuadrados y 3 dormitorios.*

5.2 Fundamento teórico

La regresión lineal múltiple generaliza la ecuación (10) a varios predictores:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_k x_{ik} + \varepsilon_i. \tag{14}$$

En este ejercicio, y es el precio de venta, x_1 son los pies cuadrados y x_2 es el número de dormitorios. El modelo se escribe como:

$$\text{precio}_i = \beta_0 + \beta_1 \text{ pies}_i + \beta_2 \text{ dormitorios}_i + \varepsilon_i. \tag{15}$$

) Los coeficientes se estiman por mínimos cuadrados ordinarios minimizando:

$$\text{SCE} = \sum_{n}^{1} \hat{} \quad \hat{} \quad \hat{2} \quad 2 \tag{16}$$

$$y_i - \beta_0 - \beta_1 x_{i1} - \beta_2 x_{i2} \ .$$

i=1

La interpretación de cada pendiente es *ceteris paribus*: $\hat{\beta}_1$ estima el cambio promedio en y ante un aumento de una unidad en x_1, manteniendo constante x_2. De

forma análoga, $\hat{\beta}_2$ estima el cambio promedio en y ante un aumento de una unidad en
x_2, manteniendo constante x_1.

El R^2, el R^2 ajustado, el error estándar residual, la prueba F global y las pruebas t individuales se interpretan de manera similar al caso simple. La prueba F contrasta la hipótesis nula de que todos los coeficientes de las pendientes son cero. Las pruebas t permiten evaluar la contribución individual de cada predictor en presencia de los demás.

Una predicción puntual se obtiene sustituyendo los valores de los predictores en la ecuación estimada:

$$\hat{y}_0 = \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \hat{\beta}_2 x_{02}. \tag{17}$$

La función predict de R calcula esa predicción. Por defecto, para un modelo lm, devuelve la estimación puntual; no devuelve un intervalo de confianza ni un intervalo de predicción salvo que se soliciten explícitamente.

5.3 Descripción del script

El archivo ejercicio-4.R ajusta el modelo múltiple y calcula la predicción solicitada:

```
modelo_ex4 <- lm(precio ~ pies + dormitorios, data = datos)
summary(modelo_ex4)

nueva_casa_ex4 <- data.frame(pies = 2000, dormitorios = 3)

prediccion_ex4 <- predict(modelo_ex4, newdata = nueva_casa_ex4)
print(prediccion_ex4)
```

Listado 15: Modelo y predicción del Ejercicio 4.

El funcionamiento es el siguiente:

1. lm(precio ~ pies + dormitorios, data = datos) ajusta el modelo de la ecuación (14) con dos predictores.
2. summary(modelo_ex4) muestra los coeficientes, errores estándar, valores t, valores p, R^2, R^2 ajustado, error estándar residual y prueba F.
3. data.frame(pies = 2000, dormitorios = 3) crea una fila nueva

con los valores de la vivienda a predecir.

4. predict(modelo_ex4, newdata = nueva_casa_ex4) evalúa el modelo ajustado en esa fila.

5. print(prediccion_ex4) muestra la predicción puntual.

5.4 Forma de ejecución

Desde la carpeta ejercicio-4:

```
Rscript ejercicio-4.R
```

Listado 16: Ejecución del script del Ejercicio 4.

La salida completa se muestra en la Figura 4.

5.5 Resultados

La Tabla 9 presenta los coeficientes del modelo y la Tabla 10 resume la bondad de ajuste.

Tabla 9: Coeficientes del modelo de regresión múltiple del Ejercicio 4.

Término	Estimación	Error estándar	Valor *t*	Valor *p*
Intercepto	5,701.45	30,165.65	0.189	0.85280
Pies cuadrados	48.51	14.53	3.338	0.00488
Dormitorios	−2,540.39	14,985.90	−0.170	0.86781

Tabla 10: Resumen del modelo del Ejercicio 4.

Indicador	Valor
R^2	0.6465
R^2 ajustado	0.5960
Error estándar residual	22,820
Grados de libertad	14
Estadístico *F*	12.8
Valor *p* de la prueba *F*	0.0006902
Predicción para 2,000 pies y dormitorios	395,091.18

Resultado

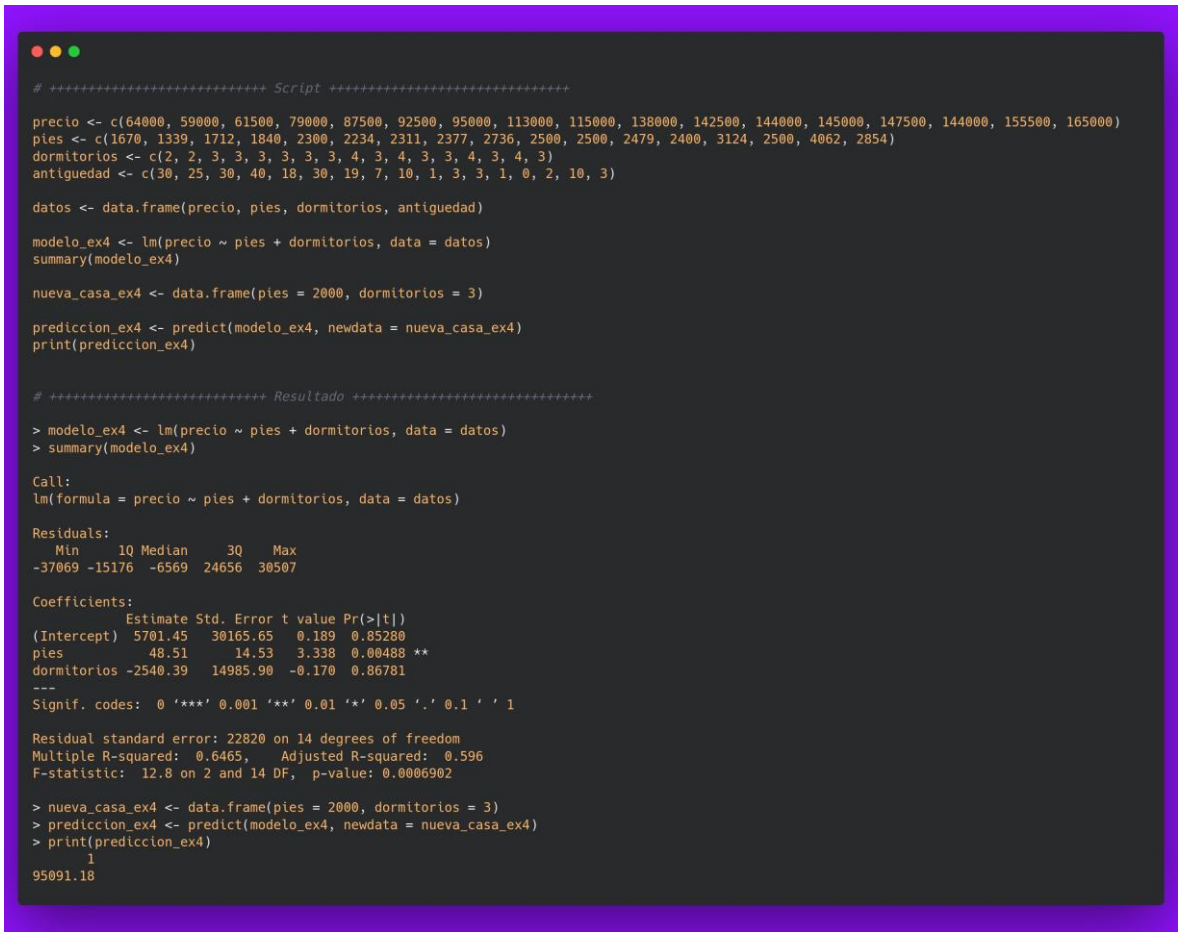
El precio de venta estimado para una casa de 2,000 pies cuadrados y 3 dormitorios es de **$95,091.18 USD**.

5.6 Interpretación

- **Pies cuadrados:** el coeficiente 48.51 indica que, manteniendo constante el número de dormitorios, un pie cuadrado adicional se asocia con un aumento

promedio de $48.51 USD en el precio. El valor p = 0.00488 es menor que 0.01, por lo que el efecto es estadísticamente significativo.

- **Dormitorios:** el coeficiente −2,540.39 no es estadísticamente significativo (p = 0.86781). En presencia de los pies cuadrados, el modelo no encuentra evidencia suficiente de un efecto lineal del número de dormitorios sobre el precio. El signo negativo no debe interpretarse como una relación causal, sino como el resultado de un coeficiente no significativo y de la correlación entre predictores.
- **Intercepto:** el valor 5,701.45 no es significativo (p = 0.85280) y corresponde a una extrapolación a cero pies cuadrados y cero dormitorios, fuera del rango observado.
- **Bondad de ajuste:** el modelo explica aproximadamente el 64.65% de la variabilidad del precio (R^2 = 0.6465). El R^2 ajustado es 0.5960, inferior al R^2 porque penaliza el predictor adicional que no aporta información significativa.
- **Comparación con el modelo simple de pies cuadrados:** al añadir dormitorios, el R^2 pasa de 0.6457 a 0.6465, un aumento muy pequeño, mientras que el R^2 ajustado disminuye de 0.6221 a 0.5960. Esto es consistente con la falta de significancia del coeficiente de dormitorios.



```
# +++++++++++++++++++++++++++ Script ++++++++++++++++++++++++++++++

precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_ex4 <- lm(precio ~ pies + dormitorios, data = datos)
summary(modelo_ex4)

nueva_casa_ex4 <- data.frame(pies = 2000, dormitorios = 3)

prediccion_ex4 <- predict(modelo_ex4, newdata = nueva_casa_ex4)
print(prediccion_ex4)


# +++++++++++++++++++++++++++ Resultado ++++++++++++++++++++++++++++++

> modelo_ex4 <- lm(precio ~ pies + dormitorios, data = datos)
> summary(modelo_ex4)

Call:
lm(formula = precio ~ pies + dormitorios, data = datos)

Residuals:
   Min     1Q Median     3Q    Max
-37069 -15176  -6569  24656  30507

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  5701.45   30165.65   0.189  0.85280
pies           48.51      14.53   3.338  0.00488 **
dormitorios -2540.39   14985.90  -0.170  0.86781
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 22820 on 14 degrees of freedom
Multiple R-squared:  0.6465,    Adjusted R-squared:  0.596
F-statistic:  12.8 on 2 and 14 DF,  p-value: 0.0006902

> nueva_casa_ex4 <- data.frame(pies = 2000, dormitorios = 3)
> prediccion_ex4 <- predict(modelo_ex4, newdata = nueva_casa_ex4)
> print(prediccion_ex4)
       1
95091.18
```

Figura 4: Salida de R para el modelo de regresión múltiple y la predicción del Ejercicio 4.

5.7 Observaciones

- El enunciado solicita ajustar el modelo en Python. El script incluido en la carpeta está escrito en R y utiliza lm y predict. El procedimiento estadístico es el mismo, pero el lenguaje difiere del solicitado.
- La predicción es una estimación puntual. El script no calcula intervalos de confianza ni de predicción, ni evalúa los supuestos del modelo.
- El coeficiente de dormitorios no significativo sugiere que, para este conjunto de datos y con pies cuadrados en el modelo, los dormitorios no aportan información adicional relevante. No debe concluirse que los dormitorios no influyan en el precio en general, sino que no hay evidencia suficiente en esta muestra y con este modelo.

6 Ejercicio 5: regresión lineal múltiple con tres predictores

6.1 Enunciado

Con base en la información del Ejercicio 3:

1. *Ajuste un modelo de regresión lineal múltiple en Python para estimar el precio de venta en función de los pies cuadrados, número de dormitorios y la antigüedad.*
2. *Estime el precio de venta de una casa de 2,000 pies cuadrados, 3 dormitorios y 10 años de antigüedad.*

6.2 Fundamento teórico

El modelo de este ejercicio amplía la ecuación (14) con un tercer predictor:

$$\text{precio}_i = \beta_0 + \beta_1\ \text{pies}_i + \beta_2\ \text{dormitorios}_i + \beta_3\ \text{antiguedad}_i + \varepsilon_i. \quad (18)$$

La interpretación de cada coeficiente es nuevamente *ceteris paribus*: por ejemplo, β_3 estima el cambio promedio en el precio ante un año adicional de antigüedad, manteniendo constantes los pies cuadrados y el número de dormitorios.

La inclusión de la antigüedad puede modificar los coeficientes de los otros predictores. Esto se relaciona con el sesgo por omisión de variables: si una variable relevante se excluye del modelo y está correlacionada con los predictores incluidos, los coeficientes estimados pueden estar sesgados. El R^2 ajustado permite comparar modelos con distinto número de predictores, ya que penaliza la complejidad adicional.

6.3 Descripción del script

El archivo ejercicio-5.R ajusta el modelo con tres predictores y calcula la predicción:

```r
modelo_ex5 <- lm(precio ~ pies + dormitorios + antiguedad, data = datos)
summary(modelo_ex5)

nueva_casa_ex5 <- data.frame(pies = 2000, dormitorios = 3, antiguedad =
    10)

prediccion_ex5 <- predict(modelo_ex5, newdata = nueva_casa_ex5)
print(prediccion_ex5)
```

Listado 17: Modelo y predicción del Ejercicio 5.

El funcionamiento es análogo al del Ejercicio 4. La diferencia es que la fórmula del modelo incluye + antiguedad y que la fila nueva incorpora antiguedad = 10. La función predict evalúa el modelo en esa fila y devuelve la estimación puntual.

6.4 Forma de ejecución

Desde la carpeta ejercicio-5:

```
Rscript ejercicio-5.R
```

Listado 18: Ejecución del script del Ejercicio 5.

La salida completa se muestra en la Figura 5.

6.5 Resultados

La Tabla 11 presenta los coeficientes del modelo y la Tabla 12 resume la bondad de ajuste.

Tabla 11: Coeficientes del modelo de regresión múltiple del Ejercicio 5.

Término	Estimación	Error estándar	Valor *t*	Valor *p*
Intercepto	82,185.649	23,008.767	3.572	0.003410
Pies cuadrados	25.941	9.583	2.707	0.017955
Dormitorios	−2,151.742	8,826.087	−0.244	0.811196
Antigüedad	−1,711.537	327.191	−5.231	0.000162

Tabla 12: Resumen del modelo del Ejercicio 5.

Indicador	Valor
R^2	0.8861
R^2 ajustado	0.8599
Error estándar residual	13,440

Grados de libertad	13
Estadístico *F*	33.72
Valor *p* de la prueba *F*	2.122×10^{-6}
Predicción para 2,000 pies, 3 dormitorios y 10 años	110,496.6

Resultado

El precio de venta estimado para una casa de 2,000 pies cuadrados, 3 dormitorios y 10 años de antigüedad es de **$110,496.6 USD**.

6.6 Interpretación

- **Pies cuadrados:** el coeficiente 25.941 indica que, manteniendo constantes los dormitorios y la antigüedad, un pie cuadrado adicional se asocia con un aumento promedio de $25.941 USD en el precio. El valor p = 0.017955 es menor que 0.05, por lo que el efecto es significativo al 5%.

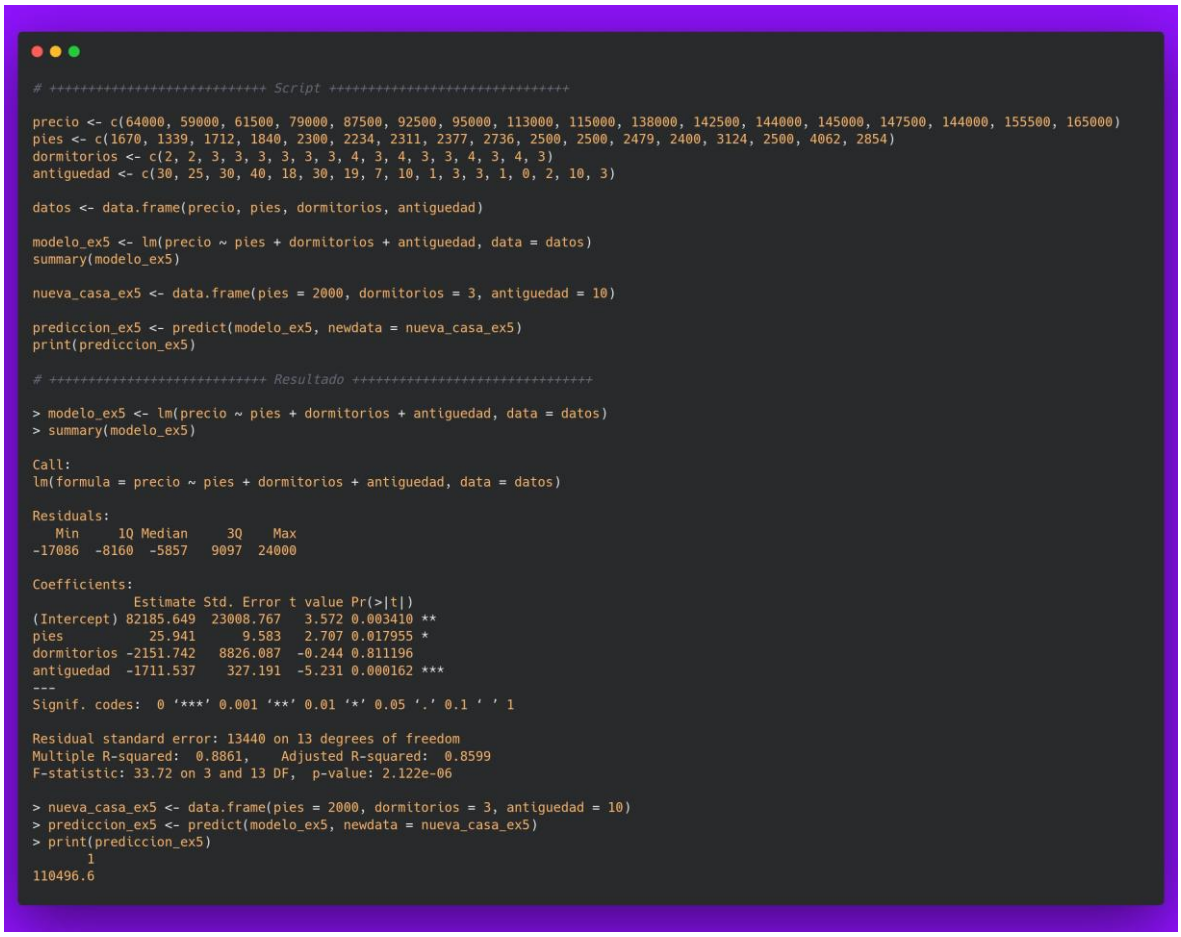
- **Dormitorios:** el coeficiente −2,151.742 no es significativo (p = 0.811196). Al controlar por pies cuadrados y antigüedad, no hay evidencia suficiente de un efecto lineal del número de dormitorios.

- **Antigüedad:** el coeficiente −1,711.537 indica que, manteniendo constantes los pies cuadrados y los dormitorios, un año adicional de antigüedad se asocia con una disminución promedio de $1,711.537 USD en el precio. El valor p = 0.000162 es menor que 0.001, por lo que el efecto es estadísticamente significativo.

- **Intercepto:** el valor 82,185.649 es significativo (p = 0.003410), pero corresponde a una vivienda de cero pies cuadrados, cero dormitorios y cero años, una combinación fuera del rango realista de los datos.

- **Bondad de ajuste:** el modelo explica aproximadamente el 88.61% de la variabilidad del precio (R^2 = 0.8861) y el R^2 ajustado es 0.8599. El error estándar residual es 13,440, inferior al de los modelos anteriores.

- **Comparación con el Ejercicio 4:** al añadir la antigüedad, el R^2 aumenta de 0.6465 a 0.8861, el R^2 ajustado aumenta de 0.5960 a 0.8599 y el error estándar residual disminuye de 22,820 a 13,440. Esto indica una mejora sustancial del ajuste.



```
# ++++++++++++++++++++++++++ Script ++++++++++++++++++++++++++++++

precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000, 115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500, 165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500, 2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10, 3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_ex5 <- lm(precio ~ pies + dormitorios + antiguedad, data = datos)
summary(modelo_ex5)

nueva_casa_ex5 <- data.frame(pies = 2000, dormitorios = 3, antiguedad = 10)

prediccion_ex5 <- predict(modelo_ex5, newdata = nueva_casa_ex5)
print(prediccion_ex5)

# ++++++++++++++++++++++++++ Resultado ++++++++++++++++++++++++++++++

> modelo_ex5 <- lm(precio ~ pies + dormitorios + antiguedad, data = datos)
> summary(modelo_ex5)

Call:
lm(formula = precio ~ pies + dormitorios + antiguedad, data = datos)

Residuals:
   Min     1Q Median     3Q    Max
-17086  -8160  -5857   9097  24000

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 82185.649  23008.767   3.572 0.003410 **
pies           25.941      9.583   2.707 0.017955 *
dormitorios -2151.742   8826.087  -0.244 0.811196
antiguedad  -1711.537    327.191  -5.231 0.000162 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 13440 on 13 degrees of freedom
Multiple R-squared:  0.8861,    Adjusted R-squared:  0.8599
F-statistic: 33.72 on 3 and 13 DF,  p-value: 2.122e-06

> nueva_casa_ex5 <- data.frame(pies = 2000, dormitorios = 3, antiguedad = 10)
> prediccion_ex5 <- predict(modelo_ex5, newdata = nueva_casa_ex5)
> print(prediccion_ex5)
       1
110496.6
```

Figura 5: Salida de R para el modelo de regresión múltiple y la predicción del Ejercicio 5.

6.7 Observaciones

- El enunciado solicita ajustar el modelo en Python. El script incluido está escrito en R; el procedimiento estadístico es equivalente, pero el lenguaje no coincide con el solicitado.
- La predicción de 110,496.6 no es directamente comparable con la de 95,091.18 del Ejercicio 4, porque la vivienda del Ejercicio 5 incluye 10 años de antigüedad y porque los coeficientes de los demás predictores cambian al incluir la antigüedad.
- La muestra es pequeña (n = 17). Los resultados deben interpretarse con cautela y no deben extrapolarse a viviendas con características muy alejadas de las observadas.
- El script no calcula intervalos de predicción ni realiza validación cruzada. La estimación es una predicción puntual del modelo ajustado.

7 Comparación, conclusiones y limitaciones

7.1 Comparación de los modelos

La Tabla 13 reúne los indicadores principales de los modelos de los Ejercicios 3, 4 y 5.

Tabla 13: Comparación de los modelos de regresión ajustados.

Ejercicio ajustado	Predictores	R^2	R^2 ajustado	Error est. residual	Predicción
3	Pies	0.6457	0.6221	22,070	—
3	Dormitorios	0.3651	0.3227	29,550	—
3	Antigüedad	0.7768	0.7619	17,520	—
4	Pies + dormitorios	0.6465	0.5960	22,820	95,091.18
5	Pies + dormitorios + antigüedad	0.8861	0.8599	13,440	110,496.6

El modelo del Ejercicio 5 presenta el mayor R^2 y el mayor R^2 ajustado, así como el menor error estándar residual. La antigüedad aporta información relevante sobre el precio en esta muestra. El número de dormitorios no resulta significativo en los modelos múltiples, mientras que los pies cuadrados sí lo son en los tres modelos en los que aparecen.

7.2 Correspondencia entre el enunciado y la implementación

La Tabla 14 resume las diferencias observadas entre lo solicitado en los enunciados y los archivos incluidos.

Tabla 14: Correspondencia entre lo solicitado y lo implementado.

Ejercicio	Solicitado	Implementado	Observación
1	Modelo en Excel y punto de equilibrio	Script en Python	Calcula el punto de equilibrio; no se incluye archivo de Excel en la carpeta.
2	R y/o Python; archivos .xls	Python y R para CSV; Python y R para PostgreSQL	Los scripts leen CSV o la base de datos; no abren directamente los .xls.
3	Estimación con los datos de viviendas		
4		Modelo múltiple en Python	

5	Modelo múltiple en Python	R	No se especifica lenguaje en el enunciado; el script usa R.
		R	El procedimiento estadístico coincide; el lenguaje difiere.
		R	El procedimiento estadístico coincide; el lenguaje difiere.

7.3 Conclusiones

1. **Ejercicio 1.** Con un precio de $40, costos fijos de $3,500 y un costo variable unitario de $26, el punto de equilibrio es de 250 unidades. En ese nivel, el ingreso y el costo total son $10,000.

2. **Ejercicio 2.** El tipo de cambio del segundo sexenio presenta una media más alta (19.5385) y una dispersión menor (varianza 2.5603, desviación estándar 1.6001) que el primero (media 16.0351, varianza 7.9107, desviación estándar 2.8126). El rango es ligeramente menor en el segundo sexenio, aunque su máximo es mayor.

3. **Ejercicio 3.** La antigüedad es el predictor simple con mayor R^2 (0.7768) y mayor R^2 ajustado (0.7619). Los pies cuadrados también presentan una relación positiva significativa (R^2 = 0.6457). Los dormitorios tienen un R^2 menor (0.3651).

4. **Ejercicio 4.** El modelo con pies cuadrados y dormitorios explica el 64.65% de la variabilidad del precio. Los pies cuadrados son significativos; los dormitorios no. La predicción para 2,000 pies cuadrados y 3 dormitorios es $95,091.18.

5. **Ejercicio 5.** El modelo con pies cuadrados, dormitorios y antigüedad explica el 88.61% de la variabilidad del precio. Los pies cuadrados y la antigüedad son significativos; los dormitorios no. La predicción para 2,000 pies cuadrados, 3 dormitorios y 10 años es $110,496.6.

7.4 Limitaciones y recomendaciones

- Los scripts de regresión no incluyen análisis de residuos, pruebas de heterocedasticidad, pruebas de normalidad ni diagnóstico de valores influyentes. Para una aplicación formal, conviene añadir estos análisis.

- Las predicciones son puntuales. No se reportan intervalos de confianza ni intervalos de predicción.

- La muestra de viviendas es pequeña (n = 17) y no se realiza validación cruzada. Los resultados pueden ser sensibles a observaciones individuales.

- En el Ejercicio 2, la varianza utilizada es poblacional. Si los datos se consideran una muestra de un proceso más amplio, debería reportarse también la varianza muestral.

- Si la entrega debe cumplir estrictamente el lenguaje solicitado, los Ejercicios 4 y 5 tendrían que reimplementarse en Python, y el Ejercicio 1 tendría que acompañarse del modelo en Excel. La Tabla 14 identifica estas diferencias.
- Las credenciales de la base de datos son de práctica. En un entorno real deben gestionarse mediante variables de entorno o un gestor de secretos.

Referencias

[1] Anderson, D. R., Sweeney, D. J. y Williams, T. A. (2008). *Estadística para administración y economía* (10.ª ed.). Cengage Learning.

[2] Montgomery, D. C., Peck, E. A. y Vining, G. G. (2012). *Introduction to Linear Regression Analysis* (5.ª ed.). John Wiley & Sons.

[3] Heizer, J. y Render, B. (2009). *Principios de administración de operaciones* (7.ª ed.). Pearson Educación.

[4] Wooldridge, J. M. (2010). *Introducción a la econometría: un enfoque moderno* (4.ª ed.). Cengage Learning.

[5] R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Versión 4.6.1.

[6] PostgreSQL Global Development Group (2026). *PostgreSQL 18 Documentation*. Disponible en la documentación oficial de PostgreSQL.

A Código fuente completo

En este apéndice se reproduce el código de los scripts documentados. Los listados se incluyen desde los archivos del proyecto. Para los dos scripts que se conectan a PostgreSQL se utiliza una copia saneada, almacenada en manual/codigo/, en la que la contraseña de práctica se sustituye por [REDACTADO]; los archivos originales permanecen en ejercicio-2. El resto de los listados se toma directamente de los archivos fuente, de modo que las correcciones se reflejen al recompilar el manual.

A.1 Ejercicio 1: ejercicio-1.py

```
s = int(input("Ingresa el precio por unidad: "))
f = int(input("Ingresa el costo fijo: "))
v = int(input("Ingresa el costo variable por unidad: "))

pequilibrio = f/(s-v)
print("El punto de equilibrio es: ", pequilibrio)
```

Listado 19: Código completo de ejercicio-1.py.

A.2 Ejercicio 2: ejercicio-2-csv.py

```python
import csv
import statistics

RUTAS = ("./sexenio-1.csv", "./sexenio-2.csv")

for ruta in RUTAS:
    with open(ruta, newline="", encoding="utf-8") as archivo:
        lector = csv.reader(archivo)
        next(lector)
        datos = [float(fila[1]) for fila in lector if fila[1] != "N/E"]

    print(f"\n{ruta}")
    print(f"Media: {statistics.mean(datos):.4f}")
    print(f"Varianza: {statistics.pvariance(datos):.4f}")
    print(f"Desviación estándar: {statistics.pstdev(datos):.4f}")
    print(f"Máximo: {max(datos):.4f}")
    print(f"Mínimo: {min(datos):.4f}")
```

```
18      print(f"Rango: {max(datos) - min(datos):.4f}")
```

Listado 20: Código completo de ejercicio-2-csv.py.

A.3 Ejercicio 2: analizar.R

```
1  rutas <- c("sexenio-1.csv", "sexenio-2.csv")
2
3  for (ruta in rutas) {
4    tabla <- read.csv(
5      ruta,
6      check.names = FALSE,
7      fileEncoding = "UTF-8"
8    )
9
10   datos <- suppressWarnings(
11     as.numeric(tabla[["Para solventar obligaciones"]])
12   )
13   datos <- datos[!is.na(datos)]
14
15   media <- mean(datos)
16   varianza <- mean((datos - media)^2)
17   desviacion <- sqrt(varianza)
18   maximo <- max(datos)
19   minimo <- min(datos)
20   rango <- maximo - minimo
21
22   cat("\n", ruta, "\n", sep = "")
23   cat(sprintf("Media: %.4f\n", media))
24   cat(sprintf("Varianza: %.4f\n", varianza))
25   cat(sprintf("Desviación estándar: %.4f\n", desviacion))
26   cat(sprintf("Máximo: %.4f\n", maximo))
27   cat(sprintf("Mínimo: %.4f\n", minimo))
28   cat(sprintf("Rango: %.4f\n", rango))
29 }
```

Listado 21: Código completo de analizar.R.

A.4 Ejercicio 2: ejercicio-2-bd.py

```
import os
import shutil
import statistics
 import subprocess

DB = {
    "host": os.getenv("DB_HOST", "localhost"),
    "puerto": os.getenv("DB_PORT", "5432"),
    "usuario": os.getenv("DB_USER", "tipo_cambio_app"),
    "password": os.getenv("DB_PASSWORD", "[REDACTADO]"),
    "base": os.getenv("DB_NAME", "tipo_cambio"),
}

 CLIENTE = shutil.which("psql")


 def consultar(sql):
    if CLIENTE is None:
        raise SystemExit("No se encontro el cliente psql.")

    comando = [
        CLIENTE,
        "--no-psqlrc",
        "--quiet",
        "--tuples-only",
        "--no-align",
        "--field-separator=\t",
        f"--host={DB['host']}",
        f"--port={DB['puerto']}",
        f"--username={DB['usuario']}",
        f"--dbname={DB['base']}",
        "--command", sql,
    ]

    entorno = os.environ.copy()
    entorno["PGPASSWORD"] = DB["password"]
    resultado = subprocess.run(
        comando,
```

```
39          capture_output=True,
40          text=True,
41          env=entorno,
42      )
43      if resultado.returncode != 0:
44          raise    SystemExit(resultado.stderr.strip())
```

```
45    return [linea.split("\t") for linea in resultado.stdout.splitlines()]
46
47
48  def mostrar_estadisticas(datos):
49  print(f"   Media: {statistics.mean(datos):.4f}")
50  print(f"   Varianza: {statistics.pvariance(datos):.4f}")
51  print(f"   Desviación estándar: {statistics.pstdev(datos):.4f}")
52  print(f"   Máximo: {max(datos):.4f}")
53  print(f"   Mínimo: {min(datos):.4f}")
54  print(f"   Rango: {max(datos) - min(datos):.4f}")
55
56
57  sexenios = consultar("SELECT id, nombre FROM sexenios ORDER BY id;")
58
59  print("Sexenios disponibles:")
60  for id_sexenio, nombre in sexenios:
61    print(f"  {id_sexenio}.  {nombre}")
62
63  opcion = input("\nElige el número del sexenio: ").strip()
64  nombres = {id_sexenio: nombre for id_sexenio, nombre in sexenios}
65  if opcion not in nombres:
66    raise SystemExit("Opción inválida.")
67
68  filas = consultar(
69    "SELECT tipo_cambio FROM tipos_cambio "
70    f"WHERE sexenio_id = {int(opcion)} AND tipo_cambio IS NOT NULL "
71    "ORDER BY fecha;"
72  )
73  datos = [float(fila[0]) for fila in filas]
74
75  print(f"\n{nombres[opcion]}")
76  print(f"      Registros: {len(datos)}")
77  mostrar_estadisticas(datos)
```

Listado 22: Código de ejercicio-2-bd.py (copia saneada).

A.5 Ejercicio 2: analizar_db.R

```
db <- list(
  host = Sys.getenv("DB_HOST", "localhost"),
  puerto = Sys.getenv("DB_PORT", "5432"),
  usuario = Sys.getenv("DB_USER", "tipo_cambio_app"),
  password = Sys.getenv("DB_PASSWORD", "[REDACTADO]"),
  base = Sys.getenv("DB_NAME", "tipo_cambio")
```

```r
)

consultar <- function(sql) {
 psql <- Sys.which("psql")
 if (nchar(psql) == 0) {
   stop("No se encontró el cliente psql.")
 }

 anterior <- Sys.getenv("PGPASSWORD", unset = NA)
 Sys.setenv(PGPASSWORD = db$password)
 on.exit({
   if (is.na(anterior)) {
     Sys.unsetenv("PGPASSWORD")
   } else {
     Sys.setenv(PGPASSWORD = anterior)
   }
 }, add = TRUE)

 argumentos <- c(
   "--no-psqlrc",
   "--quiet",
   "--tuples-only",
   "--no-align",
   "--field-separator=:",
   paste0("--host=", db$host),
   paste0("--port=", db$puerto),
   paste0("--username=", db$usuario),
   paste0("--dbname=", db$base)
 )

 salida <- system2(
   psql,
   args = argumentos,
   input = sql,
   stdout = TRUE,
   stderr = TRUE
 )

```

```
45  estado <- attr(salida, "status")
46  if (!is.null(estado) && estado != 0) {
47    stop(paste(salida, collapse = "\n"))
48  }
49  salida
50 }
51
```

```r
sexenios <- strsplit(
 consultar("SELECT id, nombre FROM sexenios ORDER BY id;"),
 ":",
 fixed = TRUE
)

ids <- vapply(sexenios, `[`, character(1), 1)
nombres <- vapply(sexenios, `[`, character(1), 2)

cat("Sexenios disponibles:\n")
for (i in seq_along(ids)) {
cat(sprintf("%s. %s\n", ids[i], nombres[i]))
}

opcion <- trimws(readline("\nElige el número del sexenio: "))
if (!(opcion %in% ids)) {
stop("Opción inválida.")
}

valores <- consultar(
paste0(
  "SELECT tipo_cambio FROM tipos_cambio ",
  "WHERE sexenio_id = ", as.integer(opcion), " ",
  "AND tipo_cambio IS NOT NULL ORDER BY fecha;"
)
)
datos <- as.numeric(valores)
datos <- datos[!is.na(datos)]

media <- mean(datos)
varianza <- mean((datos - media)^2)
desviacion <- sqrt(varianza)
maximo <- max(datos)
minimo <- min(datos)
rango <- maximo - minimo

cat("\n", nombres[match(opcion, ids)], "\n", sep = "")
 cat(sprintf("Registros: %d\n", length(datos)))
 cat(sprintf("Media: %.4f\n", media))
```

```r
cat(sprintf("Varianza: %.4f\n", varianza))
cat(sprintf("Desviación estándar: %.4f\n", desviacion))
cat(sprintf("Máximo: %.4f\n", maximo))
cat(sprintf("Mínimo: %.4f\n", minimo))
cat(sprintf("Rango: %.4f\n", rango))
```

Listado 23: Código de analizar_db.R (copia saneada).

A.6 Ejercicio 2: esquema de base_datos_postgres.sql

El archivo completo contiene el esquema, las sentencias de inserción de los 4,749 registros y los permisos de lectura. Por su extensión, aquí se reproduce el esquema de las tablas y el inicio de las inserciones. El encabezado con credenciales se omite por seguridad. El archivo completo se encuentra en ejercicio-2/base_datos_postgres.sql.

```
CREATE TABLE sexenios (
    id INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE tipos_cambio (
    id SERIAL PRIMARY KEY,
    sexenio_id INTEGER NOT NULL REFERENCES sexenios(id),
    fecha DATE NOT NULL,
    tipo_cambio NUMERIC(10,4),
    UNIQUE (sexenio_id, fecha)
);

INSERT INTO sexenios (id, nombre) VALUES
    (1, 'Sexenio 2012-2018'),
    (2, 'Sexenio 2019-2024');

-- Datos Sexenio 2012-2018
 INSERT INTO tipos_cambio (sexenio_id, fecha, tipo_cambio) VALUES
(1, '2012-01-01', 13.9787),
(1, '2012-01-02', 13.9787),
(1, '2012-01-03', 13.9476),
(1, '2012-01-04', 13.9342),
```

```
(1,                    ,
24    '2012-01-05',    13.6882)
(1,                    ,
```

Listado 24: Esquema y primeras inserciones del archivo base_datos_postgres.sql.

A.7 Ejercicio 3: ejercicio-3.R

```r
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000,
    115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500,
    165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500,
    2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10,
    3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_pies <- lm(precio ~ pies, data = datos)
summary(modelo_pies)

modelo_dorm <- lm(precio ~ dormitorios, data = datos)
summary(modelo_dorm)

modelo_antiguedad <- lm(precio ~ antiguedad, data = datos)
summary(modelo_antiguedad)
```

Listado 25: Código completo de ejercicio-3.R.

A.8 Ejercicio 4: ejercicio-4.R

```r
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000,
    115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500,
    165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500,
    2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10,
    3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_ex4 <- lm(precio ~ pies + dormitorios, data = datos)
summary(modelo_ex4)

nueva_casa_ex4 <- data.frame(pies = 2000, dormitorios = 3)
```

```r

prediccion_ex4 <- predict(modelo_ex4, newdata = nueva_casa_ex4)
print(prediccion_ex4)
```

Listado 26: Código completo de ejercicio-4.R.

A.9 Ejercicio 5: ejercicio-5.R

```r
precio <- c(64000, 59000, 61500, 79000, 87500, 92500, 95000, 113000,
    115000, 138000, 142500, 144000, 145000, 147500, 144000, 155500,
    165000)
pies <- c(1670, 1339, 1712, 1840, 2300, 2234, 2311, 2377, 2736, 2500,
    2500, 2479, 2400, 3124, 2500, 4062, 2854)
dormitorios <- c(2, 2, 3, 3, 3, 3, 3, 3, 4, 3, 4, 3, 3, 4, 3, 4, 3)
antiguedad <- c(30, 25, 30, 40, 18, 30, 19, 7, 10, 1, 3, 3, 1, 0, 2, 10,
    3)

datos <- data.frame(precio, pies, dormitorios, antiguedad)

modelo_ex5 <- lm(precio ~ pies + dormitorios + antiguedad, data = datos)
summary(modelo_ex5)

nueva_casa_ex5 <- data.frame(pies = 2000, dormitorios = 3, antiguedad =
    10)

prediccion_ex5 <- predict(modelo_ex5, newdata = nueva_casa_ex5)
print(prediccion_ex5)
```

Listado 27: Código completo de ejercicio-5.R.

B Guía de edición y compilación

B.1 Archivos del manual

- manual.tex: fuente principal del documento.
- compilar.sh: script que ejecuta latexmk y genera manual.pdf.
- generar_boxplot.py: script que regenera el diagrama de caja del Ejercicio 2 a partir de los CSV.

- codigo/: copias saneadas de los scripts de base de datos, con la contraseña sustituida por [REDACTADO].
- imagenes/boxplot_tipo_cambio.png: figura comparativa del Ejercicio 2.
- logo.png: archivo opcional para la portada. Si existe, se inserta automáticamente; si no, se muestra un recuadro de reserva.

B.2 Compilación

Desde la carpeta manual:

```
./compilar.sh
```

Listado 28: Compilación del manual con el script incluido.

También puede compilarse directamente con:

```
latexmk -pdf -interaction=nonstopmode -halt-on-error manual.tex
```

Listado 29: Compilación manual con latexmk.

El resultado es el archivo manual.pdf. Si se desea una compilación limpia, puede ejecutarse latexmk -C para eliminar los archivos auxiliares.

B.3 Sustitución de datos de portada

Antes de la entrega, buscar en manual.tex todas las apariciones de [COMPLETAR] y sustituirlas por:

- Nombre de la institución.
- Facultad, departamento o área académica.
- Materia o asignatura.
- Nombre del alumno.
- Nombre del docente.
- Carrera o grupo.
- Ciudad.

- Mes y año.

B.4 Logotipo institucional

Si se dispone de un logotipo, guardarlo como logo.png dentro de la carpeta manual. La portada lo detectará automáticamente y lo insertará en el espacio reservado. Si el logotipo tiene otra extensión, modificarse la línea correspondiente en el bloque de la portada.

B.5 Actualización de resultados

Si se modifican los scripts, deben actualizarse las tablas y el texto de resultados. Desde la carpeta examen, los comandos de ejecución son:

```
cd ejercicio-1 && python3 ejercicio-1.py

cd ejercicio-2 && python3 ejercicio-2-csv.py && Rscript analizar.R

cd ejercicio-3 && Rscript ejercicio-3.R

cd ejercicio-4 && Rscript ejercicio-4.R

cd ejercicio-5 && Rscript ejercicio-5.R
```

Listado 30: Comandos para reproducir todos los resultados del manual.

Para la variante de base de datos, primero debe cargarse la base con el archivo SQL y después ejecutar ejercicio-2-bd.py o analizar_db.R.

Si se modifican los archivos CSV del Ejercicio 2, el diagrama de caja puede regenerarse con:

```
python3 generar_boxplot.py
```

Listado 31: Regeneración de la figura del Ejercicio 2.

Si se modifican los scripts de base de datos, deben actualizarse también las copias saneadas de manual/codigo/ para que los listados del Apéndice A reflejen los cambios.

B.6 Dependencias de LaTeX

El documento utiliza paquetes habituales de TeX Live: inputenc, fontenc, lmodern, geometry, amsmath, amssymb, graphicx, booktabs, array,

multirow, float, xcolor, listings, tcolorbox, pgfplots, caption, enumitem, setspace, fancyhdr, lastpage, microtype, titlesec, textcomp y hyperref. Si falta alguno, debe instalarse desde el

gestor de paquetes de la distribución de TeX Live.

B.7 Posibles problemas

- **Imagen no encontrada:** verificar que las rutas ../ejercicio-3/ejercicio-3.png, ../ejercicio-4/ejercicio-4.png y ../ejercicio-5/ejercicio-5.png existan y que la compilación se ejecute desde la carpeta manual.
- **psql no encontrado:** instalar el cliente de PostgreSQL y asegurarse de que esté en el PATH.
- **Error de conexión a PostgreSQL:** comprobar que el servicio esté activo, que la base se haya cargado y que las variables de entorno de conexión sean correctas.
- **Caracteres acentuados en los listados:** el preámbulo incluye una tabla de conversión para los caracteres españoles. Si se añaden otros símbolos, puede ser necesario ampliarla.
- **Advertencias de compilación:** las advertencias por cajas o referencias suelen resolverse con una segunda compilación; latexmk las gestiona automáticamente.